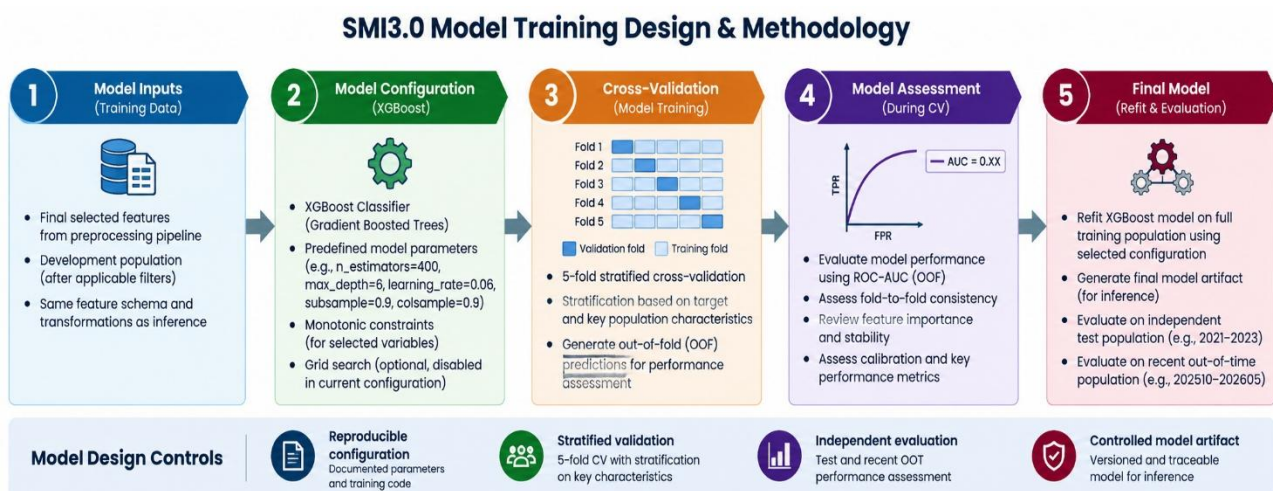


2.2 Model Design & Methodology

The validation team independently reviewed the SMI3.0 model design and training methodology through the model training code, configuration files, preprocessing implementation, model evaluation utilities, and model output logic. The review focused on the appropriateness of the modelling methodology, training and cross-validation framework, model parameterization, model-selection controls, model constraints, reproducibility, and interpretability.

SMI3.0 is implemented as a national XGBoost binary classification model. The model uses gradient-boosted decision trees to capture nonlinear relationships and interactions among the selected borrower, loan, property, credit, and macroeconomic risk characteristics. The training implementation integrates model-input preprocessing and XGBoost estimation within a common pipeline, supporting consistent application of transformations during model fitting and scoring.

2.2.1 Model Training Design & Methodology



SMI3.0 uses an XGBoost binary classification methodology. Model training is implemented using a five-fold stratified cross-validation framework, with stratification incorporating the target and selected population characteristics. Out-of-fold predictions are generated across the development population and evaluated using ROC-AUC. Following cross-validation, the model is refitted on the full training population using the selected model configuration. The resulting model is subsequently assessed on separate test and recent out-of-time populations.

The training implementation also supports monotonic constraints for selected risk variables and an optional grid-search framework for hyperparameter tuning. In the

reviewed configuration, grid search is disabled and predefined XGBoost parameters are used, including 400 estimators, maximum depth of 6, learning rate of 0.06, subsampling of 0.9, and column subsampling of 0.9.

The overall training methodology is summarized in **Fig. 2.2.1.1**, covering model inputs, model configuration, cross-validation, OOF performance assessment, and final full-sample model refit.

Conclusion: Validation reviewed the model training methodology and implementation and finds the cross-validation, full-sample refit, and independent out-of-time evaluation framework appropriately structured for model development and performance assessment.

2.2.2 Model Configuration & Hyperparameter Assessment

Validation independently reviewed the XGBoost model configuration and key hyperparameters implemented in the training pipeline. The reviewed configuration uses a binary logistic objective with ROC-AUC as the primary model discrimination metric. Key parameters are summarized below.

Parameter	Reviewed Configuration
Model	XGBoost Classifier
Objective	binary:logistic
Evaluation metric	AUC
Number of estimators	400
Learning rate	0.06
Maximum tree depth	6
Subsample	0.90
Column subsample	0.90
L2 regularization (reg_lambda)	1.0
Cross-validation folds	5
Random state	42

The configuration incorporates controls over model complexity, learning rate, observation and feature sampling, and regularization. A fixed random state is also specified to support reproducibility. The training implementation includes an optional grid-search framework covering the number of estimators, tree depth, learning rate, subsampling, and column subsampling. However, grid search is disabled in the reviewed configuration (use_grid_search=False), and the specified parameter values are therefore used directly for model training.

Conclusion: Validation independently reviewed the model configuration and hyperparameter implementation and finds the selected XGBoost configuration **appropriate for the model design and intended use**. The parameterization provides reasonable controls over model complexity and overfitting. Validation **approves the model configuration**, subject to the supporting development evidence demonstrating the basis for the final parameter selection.

Supporting Evidence: train.py, train_config.yaml, XGBoost parameter configuration, optional grid-search implementation, and model-development results.

2.2.4 Model Constraints & Design Controls

Validation reviewed the design controls incorporated into the SMI3.0 training implementation. The model configuration specifies monotonic constraints for selected risk variables to control the direction of fitted relationships where a directional risk expectation has been established.

Examples of configured constraints include increasing risk relationships for CRA_JUDGEMENT, CRA_BANKRUPTCY_FORECLOSURE_IND, DSR_DIFF_bin, and ACTLTV_bin, and decreasing relationships for CRD_SCORE_bin, NUM_OF_BOR_bin, and PRD_LOAN_PURPOSE_bin. The configuration also includes a constraint for PRD_PORT.

In addition to monotonic constraints, the training framework incorporates a fixed random state, controlled preprocessing, defined model parameters, feature exclusion controls, saved preprocessing/model artifacts, and logging of model-development outputs. These controls support consistency and reproducibility of the model-development process.

Conclusion: Validation finds the use of model constraints and associated design controls **appropriate for controlling model behaviour and supporting reproducibility**. The imposed monotonic directions should remain supported by the corresponding variable definitions, empirical relationships, and documented business rationale. Subject to this alignment, Validation **approves the model design controls**.

Supporting Evidence: train_config.yaml, monotonic-constraint configuration, train.py, preprocessing pipeline, feature-exclusion configuration, model artifact generation, and feature-selection/business-rationale assessment.

2.2.5 Model Interpretability & Explainability Assessment

Validation independently reviewed the model interpretability framework implemented for SMI3.0. Given the nonlinear tree-based structure of XGBoost, interpretability analysis is important for assessing whether model predictions are driven by economically and risk-relevant characteristics.

The reviewed training implementation includes **SHAP-based explainability analysis** on the final full-data refit model. The configuration supports SHAP sampling, identification of leading feature contributions, subgroup analysis, and feature-dependence analysis. The training process also generates feature-importance outputs.

Validation considers these analyses together with the feature-selection assessment, monotonicity review, business rationale, and observed feature behaviour. This provides a broader assessment of whether the model's principal drivers are consistent with expected mortgage-credit risk relationships rather than relying solely on statistical feature importance.

Conclusion: Validation finds the model interpretability framework **appropriate for the XGBoost methodology and intended model use**. The combination of SHAP analysis, feature importance, directional constraints, and business-rationale assessment provides an appropriate basis for understanding and challenging the principal model drivers. Validation **approves the interpretability approach**, subject to satisfactory review of the resulting feature relationships and SHAP outputs.

Supporting Evidence: train.py, SHAP configuration in train_config.yaml, shap_plots.py, shap_utility.py, feature-importance outputs, and Feature Selection Assessment.

2.6 Model Implementation, Reproducibility & Exceptions

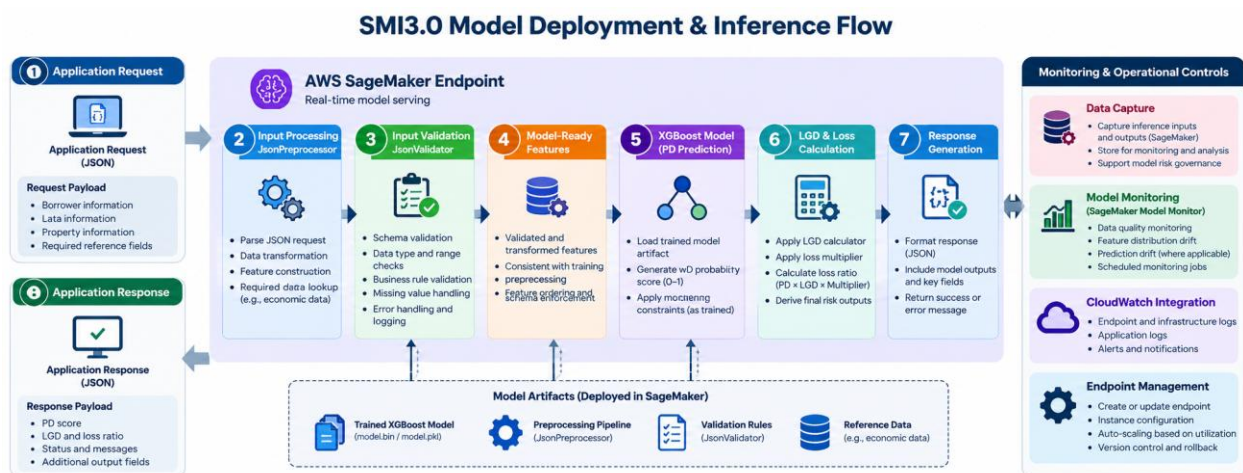
Validation reviewed the supplied model-training implementation for consistency with the documented model design and for the ability to reproduce the intended training process. The code establishes a defined sequence covering preprocessing, optional hyperparameter search, five-fold cross-validation, OOF performance calculation, full-development-sample refit, model validation outputs, SHAP analysis, and generation of the deployment model artifact.

However, validation identified implementation issues in the supplied version of train.py that require reconciliation. Specifically, the reviewed build_preprocessor() implementation references binary_monotone_cols without defining it as a function argument, while the subsequent function call attempts to provide binary_monotone_cols and contains an apparent syntax issue. The binary-category construction in the same preprocessing function also requires confirmation against the executable training version.

These observations indicate that the supplied training code should be reconciled to the exact version used to produce the final model artifact before full reproducibility can be established.

Conclusion: Validation finds the **intended model-training design and methodology appropriate**; however, Validation **does not consider implementation reproducibility fully demonstrated based on the supplied code version**. The identified code discrepancies should be reconciled against the executed/version-controlled training code and the final production model artifact. Validation approval of implementation reproducibility should therefore be **subject to closure of this observation**.

3. Model Deployment & Implementation Assessment



Validation independently reviewed the SMI3.0 deployment and inference implementation, including model artifact loading, request preprocessing and validation, model scoring, response generation, endpoint deployment, data capture, monitoring configuration, and endpoint scaling controls.

SMI3.0 is deployed through an **AWS SageMaker endpoint**. The production inference process receives application-level JSON requests, applies the configured preprocessing and validation logic, generates the required model inputs, performs model scoring using the deployed XGBoost model, applies the downstream loss-related calculations, and returns the resulting response in the required output format.

The deployment implementation separates model-serving functionality into defined stages, supporting traceability from the incoming request through preprocessing, model execution, and response generation.

Model Deployment Flow

I recommend a visually appealing deployment diagram here:

Application Request (JSON)

↓

SageMaker Endpoint

↓

Input Processing

JsonPreprocessor

↓

Input Validation

JsonValidator

↓

Model-Ready Features

↓

XGBoost Model / PD Prediction

↓

LGD & Loss Calculation

↓

Response Generation

↓

Application Response (JSON)

Then show **Data Capture / Model Monitoring / CloudWatch** as a parallel control layer around the endpoint rather than as part of the scoring sequence.

Model Artifact & Inference Implementation

Validation reviewed the model-serving implementation to assess whether the trained model artifact is appropriately loaded and used during inference. The deployment implementation includes model-loading, input-processing, prediction, and output-generation functions consistent with the SageMaker serving framework.

The business inference implementation additionally applies JsonPreprocessor and JsonValidator to transform and validate incoming application data prior to model execution. Required economic and reference information is incorporated through the configured inference preprocessing process. The resulting model-ready inputs are subsequently passed to the trained XGBoost model for prediction.

Conclusion: Validation finds the model artifact loading and inference architecture **appropriately structured for the intended production use**. The separation of preprocessing, validation, model execution, and response generation provides appropriate control points and supports traceability of production scoring. Validation **approves the inference implementation**, subject to successful reconciliation of the deployed model artifact and preprocessing configuration to the validated model version.

Deployment Controls

The reviewed deployment implementation includes controls for endpoint creation and update, model artifact configuration, instance configuration, and endpoint autoscaling. The deployment process supports updating an existing endpoint as well as creating a new endpoint where required.

The implementation also provides optional input/output data capture, allowing production inference requests and responses to be retained for subsequent monitoring and analysis. Autoscaling controls are available to adjust endpoint capacity based on configured utilization criteria.

Conclusion: Validation finds the deployment controls **appropriate for supporting controlled and scalable model serving**. Validation **approves the deployment design**, provided that production access, versioning, change-management, and infrastructure controls are maintained through the applicable technology governance processes.